

2^{ème} partie

Aller plus loin avec PHP

Gestion d'une base de données	3
Création d'une base MySql	3
• Création d'une base de données MySql	3
• Enregistrer des données	5
Interrogation d'une base MySql	5
• Ouverture de la connexion sur la base de données : mysql_connect()	5
• Sélectionner une base de données MySQL : mysql_select_db()	6
• Création et exécution de la requête : mysql_query()	6
• Compter le nombre d'enregistrements retournés : mysql_num_rows()	7
• Retour de la ligne de résultats et affichage des résultats : mysql_fetch_...()	7
• Libérer la mémoire : mysql_free_result()	9
• Fermer la connexion : mysql_close()	9
• Résumé : exemple complet	10
• Quelques autres fonctions MySQL	10
Accès à d'autres types de bases	12
• par des fonctions internes à PHP	12
• par une connexion ODBC	12
Accès par ODBC	13
• Création d'une entrée ODBC	13
• Ouverture de la connexion sur la base de données : odbc_connect()	13
• Création et exécution d'une requête : odbc_exec() ou odbc_do()	14
• Retour de la ligne de résultats : odbc_fetch_row() et affichage des résultats : odbc_result ()	14
• Affichage de tous les résultats dans un tableau HTML : odbc_result_all()	15
• Libérer la mémoire : odbc_free_result()	15
• Fermer la connexion : odbc_close()	15
• Résumé : 2 exemples complets	16
• Quelques autres fonctions ODBC	17

Les Sessions avec Php 4	19
A quoi servent les sessions ?	19
Comment fonctionnent les sessions ?	19
• Du côté du serveur	19
• Les fonctions utilisées	20
• Transmettre l'id de session	21
• Sessions et programmation orientée objet	22
A quoi ne servent pas les sessions ?	23
Quelques erreurs communes	23
Conclusion	24
Fonctions PHP	25
Gestion des chaînes de caractères : quelques fonctions	25
Fonctions mathématiques	26

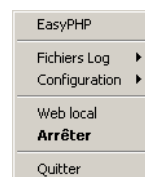
ENITA Bordeaux
U.F. Informatique

Gestion d'une base de données

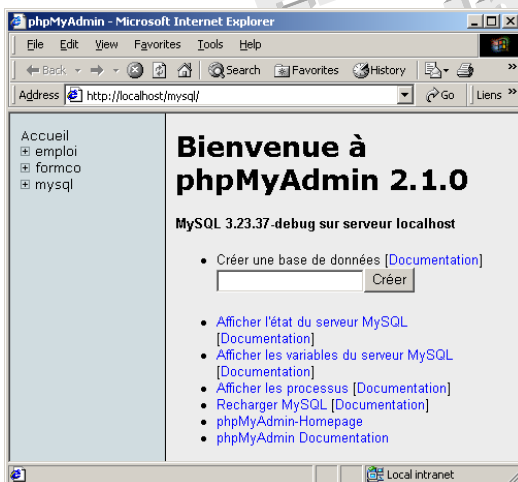
Dans ce document, nous ne parlerons pas de la conception d'une base de données : ce thème est abordé dans un autre cours. Seront traités d'abord la création d'une base de données de type **MySQL** et de ses tables par le module **PHPmyAdmin** (fourni dans **EasyPhp**) puis la connexion à une base existante, l'interrogation et la mise à jour des données, via des requêtes SQL MySQL dans une page PHP. Enfin, nous verrons comment se connecter à d'autres bases de données telles que Access, Oracle, ...

Création d'une base MySQL

La création de la base ainsi que celles des tables sera faite à l'aide de **PhpmyAdmin**. Pour démarrer ce programme, faire clic droit sur , l'icône de EasyPhp, puis choisir "Configuration" et "PhpmyAdmin".



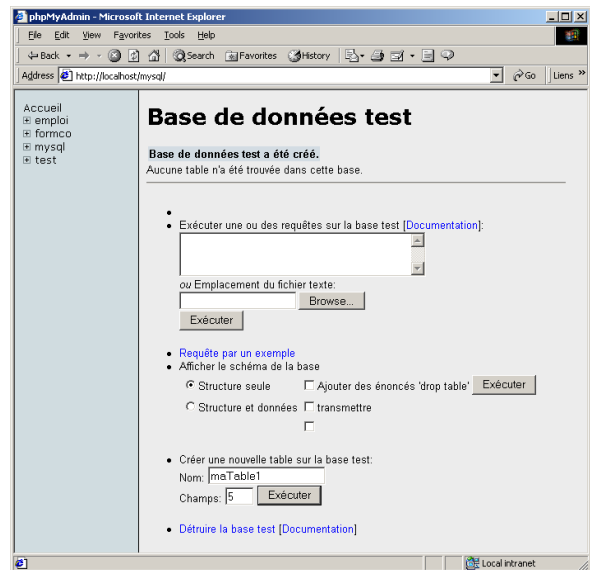
• Création d'une base de données MySQL



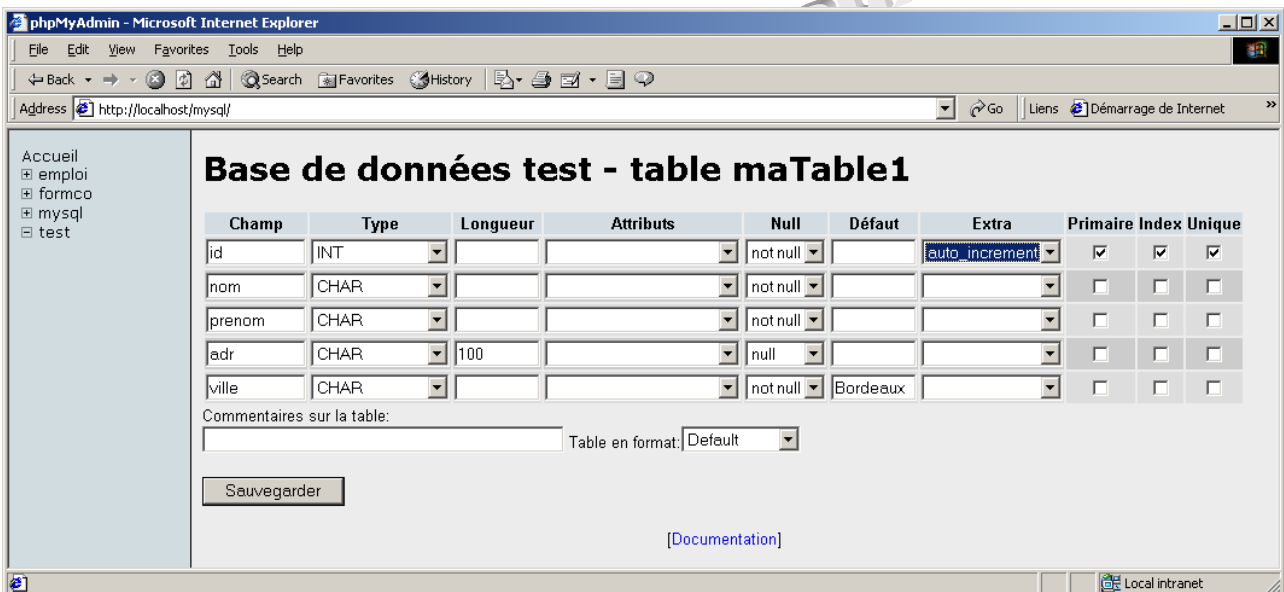
Le navigateur s'ouvre proposant :

- de créer une nouvelle base de données
- ou de choisir parmi les bases de données déjà créées (liste dans le cadre de gauche)

Lorsque la base est créée (ici, elle s'appelle "test"), la création d'une table est possible :
 Dans la rubrique, "**créer une nouvelle table sur la base test**", donner le "**nom**" de la table et le nombre de "**champs**" qu'elle contient. Cette valeur peut être modifiée ultérieurement.
"Exécuter" la commande



Voici l'écran suivant :



On y retrouve les 5 lignes qui vont décrire les champs de la table, elles-mêmes constituées de différentes colonnes.

- **Champ** = nom du champ
- **Type** = type de la donnée (numérique, alphanumérique, date, ...)
- **Longueur** = longueur de la données (adr sur 100 caractères)
- **Attributs** = type de valeurs (binaire, non signé, ...)
- **Null** = la valeur peut être nulle ou pas
- **Défaut** = valeur par défaut
- **Extra** = numérotation automatique (auto_increment)
- **Primaire** = clé de la table (automatiquement indexée et unique)
- **Index** = lors d'une recherche selon ce champ, permet un accès plus rapide aux enregistrements
- **Unique** = les doublons de ce champ sont interdit dans l'ensemble de la table

- Enregistrer des données

Lorsque la table et les champs sont créés, il est alors possible de saisir des données en vue de l'interroger.

Interrogation d'une base MySql

Pour interroger une base de données, il est impératif de maîtriser le langage SQL. Nous ne le détaillerons pas dans ce manuel.

L'accès aux données se décompose en plusieurs étapes :

- ouvrir la connexion sur le serveur MySql
- ouvrir une connexion sur la base de données
- définir la requête à exécuter
- exécuter la requête. On obtient les lignes de résultats dans un recordset.
- extraire les données de chaque ligne de résultat
- traiter les données (calcul et affichage par exemple)
- fermer le recordset pour libérer la mémoire
- fermer la connexion

- Ouverture de la connexion sur la base de données : `mysql_connect()`

La fonction qui ouvre une connexion à un serveur MySQL, est `mysql_connect()` dont la syntaxe est la suivante :

```
int mysql_connect (string hostname :port , string username , string password)
avec :      hostname : nom de l'hôte ou du site (chaîne de caractères)
           username : nom de connexion à la base (=login) (chaîne de caractères)
           password : mot de passe associé au login (chaîne de caractères)
```

La fonction retourne un nombre positif si la connexion a réussi sinon elle retourne FALSE (=0).

La connexion sera fermée automatiquement dès que l'exécution du script sera terminée, à moins d'être fermée explicitement avec `mysql_close()`, ce qui est fortement conseillée.

```
// connexion locale, login "root" et pas de mot de passe
$maConnexion = mysql_connect ("localhost", "root", "");
if ($maConnexion == FALSE)    // si la connexion a échoué
{
    die ("connexion impossible"); // affiche le message puis
                                // arrête le script
}
echo ("connexion réussie");    // ou instructions diverses
```

✓ **Autre commande : `mysql_pconnect ()`**

La fonction `mysql_pconnect()` se comporte exactement comme `mysql_connect()` mais :

- lors de la connexion, la fonction essaie de trouver une connexion permanente déjà ouverte sur cet hôte, avec le même nom d'utilisateur et de mot de passe. Si une telle connexion est trouvée, son identifiant est retourné, sans ouvrir de nouvelle connexion.

- la connexion au serveur MySQL ne sera pas terminée avec la fin du script. Au lieu de cela, le lien sera conservé pour un prochain accès (la fonction `mysql_close()` ne terminera pas une connexion persistante)

- Sélectionner une base de données MySQL : `mysql_select_db()`

Cette fonction permet d'indiquer dans quelle base se trouvent les données à traiter.

```
int mysql_select_db (string database, resource link_identifier )
avec      database : nom de la base de données MySql qui contient les tables
          link_identifier : identifiant de la connexion ouverte par mysql_connect(). C'est un
                           paramètre facultatif
```

La fonction retourne TRUE en cas de succès, FALSE en cas d'échec. Toutes les requêtes exécutées avec `mysql_query()` seront faites avec la base de données active.

```
$ConnectBase = mysql_select_db ("maBase", $maConnexion);
if ($ConnectBase == FALSE)    // si la base est inaccessible
{
    die ("base inaccessible"); // arrêt du script
}
echo ("accès correct à la base");
```

- Création et exécution de la requête : `mysql_query()`

La fonction `mysql_query()` envoie une requête SQL à la base de données active. Sa syntaxe est la suivante :

```
resource mysql_query (string query, resource link_identifier )
avec :      query : nom de la requête ou chaîne de caractères contenant la requête
          link_identifier : identifiant de la connexion ouverte par mysql_connect(). C'est un
                           paramètre facultatif
```

La fonction retourne FALSE (=0) pour indiquer l'échec de la requête, c'est-à-dire qu'elle n'a pas pu être exécutée sur le serveur, le plus souvent dans le cas d'une erreur de syntaxe Sql ou de droits insuffisants sur la base.



Remarque : Il est parfaitement possible qu'une requête correcte au niveau de sa syntaxe ne retourne aucune ligne. La fonction retourne donc TRUE.

Pour les commandes DELETE, INSERT, REPLACE, ou UPDATE, la fonction `mysql_affected_rows()` permet de connaître le nombre de lignes affectées.

Pour les commandes SELECT, `mysql_query()` retourne un identifiant de résultat que l'on peut passer à `mysql_result()`. A la fin du traitement, il est conseillé de libérer la mémoire avec `mysql_free_result()`.

```
$sql = "Select * from maTable"; // La requête
$result = mysql_db_query ("maBase", $sql);    // envoi au serveur
if ($result == FALSE)    // si la requête est incorrecte
{
    die ("requête incorrecte : $sql"); // arrêt du script
}
echo ("requête correcte");
```

✓ Autre commande

La fonction `mysql_db_query()` permet tout à la fois d'indiquer de quelle base sont extraites les données et envoie une requête SQL à la base de données donnée en paramètre. Sa syntaxe est la suivante :

```
resource mysql_db_query (string database, string query, resource link_identifieur )
avec :      database : nom de la base de données qui contient les tables
            query : nom de la requête ou chaîne de caractères contenant la requête
            link_identifieur : identifiant de la connexion ouverte par mysql_connect(). C'est un
                        paramètre facultatif
```

- Compter le nombre d'enregistrements retournés : mysql_num_rows()

La fonction `mysql_num_rows()` retourne le nombre d'enregistrements qui répondent à la requête.

```
int mysql_num_rows (resource result_identifieur)
avec      result_identifieur : identifiant retourné par mysql_query().
```



Remarque : Cette commande n'est utile que pour les commandes SELECT. Pour connaître le nombre d'enregistrements traités par INSERT, UPDATE ou DELETE, ce sera `mysql_affected_rows()`

```
$result = mysql_query("SELECT nom, prenom FROM maTable");
$nb = mysql_num_rows($result);
echo "il y a $nb enregistrements";
```

- Retour de la ligne de résultats et affichage des résultats : mysql_fetch ...()

Php possède différentes fonctions qui permettent le retour des résultats issus d'une requête SQL. L'une permettra d'afficher les résultats selon la position du champ dans la requête (indice), une autre selon le nom du champ donné dans la table et une dernière qui permettra tout autant de travailler sur les indices que sur les noms des champs. La manière d'afficher le résultat dépendra de la fonction utilisée.

- ✓ à partir des numéros de champ : `mysql_fetch_row()`

La fonction `mysql_fetch_row()` retourne UNE ligne d'enregistrements appelée **recordset** sous la forme d'un tableau. Chaque colonne est enregistrée sous la forme d'un tableau commençant à la position 0.

```
array mysql_fetch_row (resource result_identifieur )
avec :      resource_identifieur : identifiant retourné par mysql_query().
```

Cette fonction retourne un tableau énuméré qui correspond à la ligne demandée, ou FALSE, s'il ne reste plus de ligne. Comme la fonction ne retourne qu'une seule ligne, il faudra boucler sur le nombre d'enregistrements précédemment calculé par la fonction `mysql_num_rows()`.

On affiche les résultats comme étant les éléments d'un tableau.

```
// boucle sur le nbre de lignes de $result
for ($i = 0 ; $i < mysql_num_rows($result) ; $i++)
{
    // extrait de $result la prochaine ligne sous la forme d'un tableau
    énuméré
    $tbl_ligne = mysql_fetch_row ($result);
    echo ("{$tbl_ligne[0]} -- {$tbl_ligne[1]<BR>"); // affichage
}
```

✓ à partir des noms de champ : `mysql_fetch_object()`

La fonction `mysql_fetch_object()` retourne UNE ligne d'enregistrements appelée **recordset**. cette fonction est identique à `mysql_fetch_array()`, à la différence qu'elle retourne un objet à la place d'un tableau. On peut alors accéder aux valeurs des champs par leur nom, mais plus par leur indice.

```
array mysql_fetch_object (resource result_identifier, int result_type )
avec :          resource_identifier : identifiant retourné par mysql_query()
               result_type : constante facultative qui peut être MYSQL_ASSOC,
               MYSQL_NUM ou MYSQL_BOTH
```

Cette fonction retourne une seule ligne sous forme d'un objet dont les propriétés correspondent à une ligne de résultat, ou FALSE, s'il ne reste plus de ligne. Il faudra boucler sur le nombre d'enregistrements précédemment calculé par la fonction `mysql_num_rows()` ou tester s'il ne reste plus de ligne.

Pour afficher les résultats, on donne le nom de l'objet suivi du nom du champ dans la base et séparé par ->

```
// boucle tant qu'il y a des lignes de résultat dans $result
while ( ($ligne = mysql_fetch_object($result)) == TRUE)
{
    echo ("{$ligne->nom} -- {$ligne->prenom<BR>"); // affichage
}
```

✓ à partir des numéros ou des noms de champ : `mysql_fetch_array()`

La fonction `mysql_fetch_array()` retourne un tableau qui contient la ligne demandée, ou FALSE s'il ne reste plus de ligne. Il s'agit une fonction étendue de `mysql_fetch_row()` car elle enregistre les données dans un tableau à indices numériques et/ou dans un tableau à indices alphanumériques.

```
array mysql_fetch_array (resource result_identifier, int result_type )
avec :          resource_identifier : identifiant retourné par mysql_query()
               result_type : constante facultative qui déterminent l'accessibilité des données :
               MYSQL_ASSOC : dans tableau à indices alphanumériques uniquement
               MYSQL_NUM : dans tableau à indices numériques uniquement
               ou MYSQL_BOTH : on pourra utiliser l'un ou l'autre des indices
```



Remarque : Si plusieurs colonnes portent le même nom, il faudra les distinguer par un alias.
exemple : `select t1.nom as n1, t2.nom as n2 from t1, t2;`

Comme les 2 fonctions précédentes, cette fonction retourne une seule ligne. Il faudra boucler sur le nombre d'enregistrements précédemment calculé par la fonction `mysql_num_rows()` ou tester s'il ne reste plus de ligne.

Pour afficher les résultats, on utilise la syntaxe des tableaux à indices alphanumériques, les indices correspondant aux noms des champs dans les tables.

```
while ( ($ligne = mysql_fetch_array($result)) == TRUE)
{
    echo $row["nom"];
    echo $row["prenom"];
}
```

✓ Quelle fonction choisir ?

Concernant la vitesse, `mysql_fetch_object()` est aussi rapide que `mysql_fetch_array()` et presque aussi rapide que `mysql_fetch_row()` (la différence est insignifiante). Etant donné les possibilités plus grandes apportées par `mysql_fetch_array()` sans perte de performance, il est conseillé d'utiliser cette fonction.

• Libérer la mémoire : `mysql_free_result()`

La fonction `mysql_free_result()` efface le résultat de la mémoire pour éviter une saturation de la place mémoire lors de l'exécution du script.

```
int mysql_free_result (resource result_identifieur)
avec :      resource_identifieur : identifiant retourné par mysql_query()
```

```
mysql_free_result ($result);
```

• Fermer la connexion : `mysql_close()`

La fonction `mysql_close()` ferme la connexion MySQL. Elle retourne FALSE si la connexion a échoué et TRUE en cas de succès.

```
boolean mysql_close (resource link_identifieur )
avec      link_identifieur : identifiant de la connexion ouverte par mysql_connect().
```



Remarques :

- cette commande n'est pas obligatoire, car toutes les connexions non persistantes seront automatiquement fermées à la fin du script. Toutefois elle est fortement conseillée.
- `mysql_close()` ne ferme pas les connexions persistantes générées par `mysql_pconnect()`.

```
mysql_close ($maConnexion);
```

- Résumé : exemple complet

```

<?
$host="localhost";           // nom de l'hôte
$user="root";                // nom de login
$password="";                // mot de passe
$db="emploi";                // nom de la base
$sql = " select * from offre; // requête

// connexion au serveur
$maConnexion = mysql_connect ($host, $user, $password);
if ($maConnexion == FALSE)
{
    die ("La connexion a échoué");
}

// déclaration du nom de la base
$ConnectBase = mysql_select_db ("emploi", $maConnexion);
if ($ConnectBase == FALSE) // si la base est inaccessible
{
    die ("base inaccessible"); // arrêt du script
}

// envoi au serveur le nom de la requête
$result = mysql_query ($sql);
if ($result == FALSE) // si la requête est incorrecte
{
    die ("requête incorrecte : $sql"); // arrêt du script
}

// boucle sur les d'enregistrements retournés
while (($ligne = mysql_fetch_array($result) == TRUE)
{
    // affichage des 2 champs "nom_entreprise" et "detail_offre"
    echo "<H3>" . $ligne["nom_entreprise"] .
        " -- " . $ligne["detail_offre"] . "</H3>";
}
// libération de l'espace mémoire pris par les recordsets
mysql_free_result ($result);
// libération de la connexion au serveur MySQL
mysql_close ($maConnexion);
?>

```

- Quelques autres fonctions MySQL

fonction	rôle et syntaxe
mysql_get_host_info	Lit les informations sur l'hôte MySQL. string mysql_get_host_info (resource link_identifiant)
mysql_get_server_info	Lit les informations sur le serveur MySQL. string mysql_get_server_info (resource link_identifiant)

mysql_fetch_lengths	Retourne la taille de chaque colonne d'une ligne de résultat. array mysql_fetch_lengths (resource result_identifiant)
mysql_insert_id	Retourne l'identifiant généré par la dernière requête INSERT. int mysql_insert_id (resource link_identifiant)
mysql_create_db	Crée une base de données MySQL. int mysql_create_db (string database_name , resource link_identifiant)
mysql_change_user	Change le nom de session de l'utilisateur actif. int mysql_change_user (string user , string password , string database , resource link_identifiant)
mysql_field_len	Retourne la longueur du champs spécifié. int mysql_field_len (resource result_identifiant , int field_offset)
mysql_list_fields	: Liste les champs du résultat MySQL. resource mysql_list_fields (string database_name , string table_name , resource link_identifiant)
mysql_list_dbs	Liste les bases de données disponibles sur le serveur MySQL. resource mysql_list_dbs (resource link_identifiant)
mysql_num_fields	Retourne le nombre de champs d'un résultat. int mysql_num_fields (resource result_identifiant)
mysql_db_name	Lit les noms des bases de données. int mysql_db_name (resource result_identifiant , int row , mixed field)
mysql_error	Retourne le texte associé avec l'erreur générée lors de la dernière requête. string mysql_error (resource link_identifiant)
mysql_affected_rows	Retourne le nombre de lignes affectées lors de la dernière requête SQL. (utilisée dans le cas des requêtes de manipulation des données (insert, update, delete, ...)) int mysql_affected_rows (resource link_identifiant)
mysql_field_type	Retourne le type de la colonne spécifiée dans le résultat courant. string mysql_field_type (resource result_identifiant , int field_offset)
mysql_field_seek	Déplace le pointeur de résultat. int mysql_field_seek (resource result_identifiant , int field_offset)
mysql_list_tables	Liste les tables d'une base de données. resource mysql_list_tables (string database , resource link_identifiant)
mysql_tablename	Lit le nom de la table qui contient le champs spécifié. string mysql_tablename (resource result_identifiant , int i)
mysql_field_table	: Retourne le nom de la table où se trouve une colonne. string mysql_field_table (resource result_identifiant , int field_offset)
mysql_data_seek	Déplace le pointeur interne de résultat vers un numéro d'enregistrement int mysql_data_seek (resource result_identifiant , int row_number)
mysql_drop_db	Efface une base de données MySQL. boolean mysql_drop_db (string database_name , resource link_identifiant)
mysql_field_name	Retourne le nom d'une colonne. string mysql_field_name (resource result_identifiant , int field_index)

Accès à d'autres types de bases

L'un des gros intérêts de PHP est sa facilité à accéder à diverses bases de données. PHP contient des fonctions qui permettent d'accéder aux bases de SGBD courants. Mais PHP permet aussi la connexion aux bases de données par l'intermédiaire de ODBC (Open DataBase Connectivity).

Comme pour une base de données MySQL, la procédure d'accès aux données se fait en plusieurs étapes :

- ouvrir une connexion sur la base de données via ODBC
- définir la requête à exécuter
- exécuter la requête. On obtient les lignes de résultats dans un recordset.
- extraire les données de chaque ligne de résultat
- traiter les données (calcul et affichage par exemple)
- fermer le recordset pour libérer la mémoire
- fermer la connexion

Les fonctions étant quasiment identiques à celles décrites dans la partie précédente, nous ne détaillerons que les différences.

• par des fonctions internes à PHP

PHP contient des fonctions internes à certaines bases. Par exemple :

Oracle, Oracle 8, SQLServer, dBase, filePro, Informix, mSQL, Sybase, PostgreSQL **et bien sûr MySQL que nous avons déjà vu.**

Exemples de fonctions internes :

pour Sybase : `sybase_connect()`, `sybase_fetch_array()`, `sybase_close()`, `sybase_result()`, ...
pour Oracle : `ora_logon()`, `ora_exec()`, `ora_fetch()`, `ora_logoff()`, ...

• par une connexion ODBC

La grande différence avec l'utilisation des fonctions natives de PHP, est qu'il est nécessaire d'avoir le pilote ODBC du type de base à laquelle on veut accéder, puis de créer le lien entre la base et l'application. Nous allons le détailler dans le chapitre suivant.

✓ Qu'est-ce qu'une connexion ODBC ?

ODBC signifie **Open DataBase Connectivity**. Il s'agit d'un format défini par Microsoft permettant la communication entre des clients fonctionnant sous Windows et les SGBD du marché.

La technologie ODBC permet d'interfacer de façon standard une application à n'importe quel serveur de bases de données, pour peu que celui-ci possède un driver ODBC (la quasi-totalité des SGBD possèdent un tel pilote, dont tous les principaux SGBD du marché).

Bien que ODBC permette un interfaçage avec des bases de données indépendamment du SGBD, cette technologie reste une solution propriétaire de Microsoft.

Cela se traduit par une dépendance de la plateforme (ODBC ne fonctionne que sur les plateformes Microsoft Windows).

Accès par ODBC

• Création d'une entrée ODBC

Elle doit se faire sur le serveur Web. Le gestionnaire ODBC est présent sur les systèmes Windows 3.1, 9x, 2000 sous l'icône suivante :



- sous Windows 2000, la succession de menus est : **Démarrer / Paramètres / Panneau de configuration / Outils d'administration/ Sources de données (ODBC)**
- sous Windows NT 4, la succession de menus est : **Démarrer / Paramètres / Panneau de configuration / Sources de données ODBC**
- Cliquer sur l'onglet "**sources de données système**" puis le bouton "**Ajouter...**"
- Sélectionner le pilote correspondant à la base de données.
Exemple : Microsoft Access Driver
- Cliquer sur "**Terminer**"
- Dans la boîte de dialogue qui apparaît :
 - saisir le "**nom de la source de données**". C'est ce nom qui servira de lien entre l'application et la base de données.
Exemple : eval
 - La zone "description" est libre. Il s'agit de commentaires.
 - Il reste à indiquer l'endroit où se trouve la base de données. Pour cela, cliquer sur "**Sélectionner...**". Indiquer le **chemin d'accès à la base** qui peut être située sur le serveur Web ou sur une machine distante.
- L'entrée ODBC est créée.

• Ouverture de la connexion sur la base de données : `odbc_connect()`

La fonction `odbc_connect ()` permet de se connecter à une source de données. Elle retourne un identifiant qui sera nécessaire pour toutes les autres fonctions. En cas d'échec, la valeur sera FALSE (=0).

`int odbc_connect (string dsn, string user, string password, int cursor_type)`
 avec :
 dsn : nom de la source de données ODBC (voir ci-dessous) (chaîne de caractères)
 user : nom de connexion à la base (=login) (chaîne de caractères)
 password : mot de passe associé au login (chaîne de caractères)
 cursor_type : constante qui peut prendre comme valeurs
 SQL_CUR_USE_IF_NEEDED, SQL_CUR_USE_DRIVER, SQL_CUR_DEFAULT,
 SQL_CUR_USE_ODBC. Cette dernière valeur peut résoudre certains
 problèmes liés aux pointeurs entre autre.

```

$dsn = "eval";
$maConnexion = odbc_connect ($dsn, "administrateur", "");
if ($maConnexion == FALSE) // si la connexion a échoué
{
    die ("échec à la connexion); // fin du script
}

```

- Création et exécution d'une requête : odbc_exec() ou odbc_do()

Les 2 fonctions `odbc_exec()` et `do_exec()` sont strictement équivalentes et envoient une commande SQL à la source de données. Sa syntaxe est la suivante :

resource `odbc_exec` (resource `connection_id`, string `query_string`)
 avec : `connection_id` : identifiant de la connexion ouverte par `odbc_connect()`.
 `query` : nom de la requête ou chaîne de caractères contenant la requête

La fonction retourne FALSE (=0) pour indiquer l'échec de la requête, c'est-à-dire qu'elle n'a pas pu être exécutée sur le serveur, le plus souvent dans le cas d'une erreur de syntaxe Sql ou de droits insuffisants sur la base.



Remarque : Il est parfaitement possible qu'une requête correcte au niveau de sa syntaxe ne retourne aucune ligne. La fonction retourne donc TRUE.

```

$sql = "Select * from maTable"; // La requête
$result = odbc_exec ($maConnexion, $sql); // envoi au serveur
if ($result == FALSE) // si la requête est incorrecte
{
    die ("requête incorrecte : $sql"); // arrêt du script
}
echo ("requête correcte");

```

- ✓ **Autres fonctions : odbc_prepare () et odbc_execute ()**

Elle sont utilisées pour les exécutions multiples de requêtes SQL.

- Retour de la ligne de résultats : odbc_fetch_row()
et affichage des résultats : odbc_result ()

La fonction `odbc_fetch_row()` retourne UNE ligne d'enregistrements appelée **recordset**.

int `odbc_fetch_row` (resource `result_id`, int `row_number`)
 avec : `result_id` : identifiant retourné par `odbc_exec()`
 `row_number` : numéro de ligne à lire (entier facultatif)

Cette fonction retourne FALSE s'il ne reste plus de ligne. Comme la fonction ne retourne qu'une seule ligne, il faudra boucler sur le nombre d'enregistrements.

On lit les données de résultats grâce à la fonction `odbc_result()`.

string `odbc_result`(`odbc_prepare result_id`, mixed `field`)
 avec : `result_id` : identifiant retourné par `odbc_exec()`
 `field` : numéro de colonne du champ (numérique entier : valeur mini = 1)
 ou nom du champ (chaîne de caractères)


```
// boucle sur le nbre de lignes de $result
while (odbc_fetch_row($result) == TRUE)
{
    echo (odbc_result($result, "nom") . " -- " .
        odbc_result($result, "prenom") . "<BR>"); //affichage
    echo (odbc_result($result, 1) . " -- " . // champ "nom"
        odbc_result($result, 2) . "<BR>"); // champ "prenom"
}
```



Remarque : Pour passer en revue toutes les lignes d'un résultat plusieurs fois, on peut appeler `odbc_fetch_row()` avec `row_number = 1`, puis continuer à appeler `odbc_fetch_row()` sans le paramètre `row_number` pour passer en revue tout le résultat. Si un pilote ne supporte pas la lecture des lignes par numéro, le paramètre sera ignoré.

- Affichage de tous les résultats dans un tableau HTML : `odbc_result_all()`

Php possède une fonction ODBC qui permet d'afficher tous les résultats d'une requête sous forme d'un tableau HTML ainsi que le nombre de lignes retournées... le tout en une seule ligne d'instruction.

```
int odbc_result_all (odbc_prepare result_id, string format)
avec :      result_id : identifiant retourné par odbc_exec()
           format : chaîne de caractères qui permet de modifier la présentation du tableau HTML
```

`odbc_result_all()` retourne le nombre de lignes dans le résultat, ou FALSE en cas d'erreur.

```
echo (odbc_result_all($result) . "<HR>");
```

- Libérer la mémoire : `odbc_free_result()`

La fonction `odbc_free_result()` libère les ressources associées à un résultat. Elle n'est nécessaire que si on craint d'utiliser trop de mémoire lors de l'exécution d'un script, car tous les résultats en mémoire seront libérés à la fin du script. Il est conseillé de la mettre. Elle retourne toujours la valeur TRUE.

```
int odbc_free_result (resource result_identifieur)
avec :      resource_identifieur : identifiant retourné par odbc_query()
```



Remarque : Si l'auto-validation est désactivée (voir `odbc_autocommit()`) et que `odbc_free_result()` est appelé avant de valider les requêtes, toutes les transactions préparées sont annulées

```
odbc_free_result ($result);
```

- Fermer la connexion : `odbc_close()`

La fonction `odbc_close()` ferme la connexion avec la source de données. Elle retourne FALSE si la connexion a échoué et TRUE en cas de succès.

```
boolean mysql_close (resource connection_id )
avec      connection_id : identifiant de la connexion ouverte par odbc_connect().
```



Remarques : `odbc_close()` échouera s'il y a des transactions en cours sur cette connexion. Dans ce cas, la connexion restera ouverte

```
mysql_close ($maConnexion);
```

- Résumé : 2 exemples complets

- ✓ Affichage des résultats avec boucle sur les enregistrements

```
<?
// connexion à la source de données appelée "eval"
$maConnexion = odbc_connect ("eval", "administrateur", "");
if ($maConnexion == FALSE) // si la connexion a échoué
{
    die ("échec à la connexion"); // fin du script
}

// définition et envoi d'une instruction SQL au serveur
$sql = "select nom_etud, prenom_etud from t_etudiant";
$result = odbc_exec ($maConnexion, $sql); // exécution

// boucle sur le nombre d'enregistrements retournés
while (odbc_fetch_row($result) == TRUE) // si ligne trouvée
{
    echo (odbc_result ($result, "nom_etud") . " -- " .
        odbc_result ($result, "prenom_etud") . "<BR>");
}

// libération de la mémoire
odbc_free_result($result);
// fermeture de la connexion avec la source de données
odbc_close($maConnexion);
?>
```


✓ Affichage de tous les résultats issus de la requête

```

<?
// connexion à la source de données appelée "eval"
$maConnexion = odbc_connect ("eval", "administrateur", "");
if ($maConnexion == FALSE) // si la connexion a échoué
{
    die ("échec à la connexion"); // fin du script
}

// définition et envoi d'une instruction SQL au serveur
$sql = "select nom_etud, prenom_etud from t_etudiant";
$result = odbc_exec ($maConnexion, $sql); // exécution

// affichage des résultats dans un tableau HTML
echo (odbc_result_all($result) . "<HR>");

// libération de la mémoire
odbc_free_result($result);
// fermeture de la connexion avec la source de données
odbc_close($maConnexion);
?>

```

• Quelques autres fonctions ODBC

fonction	rôle et syntaxe
odbc_pconnect	Ouvre une connexion persistante à une source de données. resource odbc_pconnect (string dsn , string user , string password , int cursor_type)
odbc_rollback	Annule une transaction. int odbc_rollback (odbc_prepare connection_id)
odbc_autocommit	Active le mode auto-validation. int odbc_autocommit (resource connection_id , int OnOff)
odbc_commit	Valide une transaction ODBC. int odbc_commit (resource connection_id)
odbc_primarykeys	Liste les colonnes utilisées dans une clé primaire. int odbc_primarykeys (resource connection_id , string qualifier , string owner , string table)
odbc_num_rows	Nombre de ligne dans un résultat. int odbc_num_rows (odbc_prepare result_id)
odbc_field_len	Lit la longueur d'un champs. int odbc_field_len (resource result_id , int field_number)
odbc_tableprivileges	Liste les tables et leurs privilèges. int odbc_tableprivileges (resource connection_id , string qualifier , string owner , string name)
odbc_gettypeinfo	Liste les types de données supportés par une source. int odbc_gettypeinfo (resource connection_id , int data_type)
odbc_statistics	Calcule des statistiques sur une table. int odbc_statistics (resource connection_id , string qualifier , string owner , string table_name , int unique , int accuracy)

odbc_error	Lit le dernier code d'erreur. string odbc_error (resource connection_id)
odbc_columns	Liste les colonnes d'une table. int odbc_columns (resource connection_id , string qualifier , string owner , string table_name , string column_name)
odbc_field_name	Lit le nom de la colonne. string odbc_field_name (resource result_id , int field_number)
odbc_execute	Exécute une requête SQL préparée. int odbc_execute (resource result_id , array parameters_array)
odbc_columnprivileges	Liste les colonnes et leurs droits associés. int odbc_columnprivileges (resource connection_id , string qualifier , string owner , string table_name , string column_name)
odbc_errormsg	Lit le dernier message d'erreur. string odbc_errormsg (int connection_id)
odbc_procedures	Liste les procédures stockées. int odbc_procedures (resource connection_id , string qualifier , string owner , string name)
odbc_foreignkeys	Liste les clés étrangères. int odbc_foreignkeys (resource connection_id , string pk_qualifier , string pk_owner , string pk_table , string fk_qualifier , string fk_owner , string fk_table)
odbc_close_all	Ferme toutes les connexions ODBC. void odbc_close_all ()
odbc_procedurecolumns	Liste les paramètres des procédures. int odbc_procedurecolumns (resource connection_id , string qualifier , string owner , string proc , string column)
odbc_field_precision	Alias de odbc_field_len. string odbc_field_precision (resource result_id , int field_number)
odbc_cursor	Lecture du pointeur de fiche courante (cursorname). string odbc_cursor (resource result_id)
odbc_setoption	Modifie les paramètres ODBC. int odbc_setoption (resource id , int function , int option , int param)
odbc_num_fields	Nombre de colonnes dans un résultat. int odbc_num_fields (resource result_id)
odbc_fetch_into	Lit une ligne de résultat, et la place dans un tableau. int odbc_fetch_into (resource result_id , int rownumber , array result_array)
odbc_prepare	Prépare une commande pour l'exécution. resource odbc_prepare (resource connection_id , string query_string)
odbc_specialcolumns	Retourne l'ensemble optimal de colonnes, qui permettent de définir uniquement une ligne dans une table. int odbc_specialcolumns (resource connection_id , int type , string qualifier , string owner , string table , int scope , int nullable)
odbc_field_type	Type de données d'un champs. string odbc_field_type (resource result_id , int field_number)
odbc_tables	Liste les tables d'une source. int odbc_tables (odbc_setoption connection_id , string qualifier , string owner , string name , string types)
odbc_field_num	Numéro de colonne. int odbc_field_num (resource result_id , string field_name)

Les Sessions avec Php 4

Article rédigé par Guillaume SMET [Avril 2001] - <http://www.phpinfo.net/imp-article.php?art=sessions>

A quoi servent les sessions ?

Lorsque qu'une personne se connecte sur votre site, elle ouvre une session. Il est parfois intéressant de conserver les valeurs de certaines variables tout au long de cette session.

Exemple : La personne entre un login et un mot de passe pour accéder au site et on souhaite vérifier au début de chaque page du site que cette personne est bien autorisée.

Solutions possibles :

On peut éventuellement transmettre la valeur de ces variables dans l'URL mais il s'agit parfois de données sensibles, de tableaux, d'objets... Ce n'est pas une solution élégante et elle n'est pas toujours réalisable. Cette solution est totalement irrecevable quand il s'agit de sécurité. De plus, une URL est limitée à 255 caractères.

On peut également enregistrer ces données en utilisant un cookie sur la machine du client. Ceci pose plusieurs problèmes : si le client n'accepte pas les cookies, ce qui est relativement courant, on ne peut stocker les données. De plus, c'est une solution client-side assez peu fiable au niveau sécurité par nature.

Aucune de ces solutions n'est réellement satisfaisante pour ce que nous souhaitons faire. De nombreux moyens de gérer les sessions ont été développés reposant en général sur des bases de données. PHP4 introduit des fonctions permettant de gérer les sessions relativement simplement. C'est l'objet du présent article.

Comment fonctionnent les sessions ?

- Du côté du serveur

Lors de l'ouverture d'une session, une id de session est attribuée au client. Celle-ci permet de l'identifier et d'accéder à ses variables de session.

Les **variables de session** sont enregistrées sur le serveur dans le répertoire `session.save_path` défini dans la configuration de PHP sous la forme d'un fichier texte par session.

A l'aide de l'id de session de l'utilisateur, PHP peut retrouver la valeur des variables de session dans le fichier texte correspondant.

Simplifions : Une personne se connecte avec son login et son mot de passe. On lui donne l'id de session 1. On enregistre son login et son mot de passe comme variables de session. Sur le serveur, un fichier texte session_1 est créé. Il contient login="login rentré"; password="mot de passe rentré". C'est une simplification assez grossière mais n'hésitez pas à aller voir à quoi ressemble un tel fichier dans le répertoire session.save_path (configuration de PHP).

On comprend assez vite la nécessité de transmettre l'id de session de l'utilisateur de page en page. Cette question sera abordée au paragraphe 3.

• Les fonctions utilisées

PHP4 dispose d'un certain nombre de fonctions permettant de manipuler les sessions. Voici les plus utiles :

- **session_start** : cette fonction est à mettre en tête de toutes les pages où vous souhaitez avoir accès à vos variables de session. Il ne doit y avoir aucun code affichable avant l'utilisation de cette fonction sous peine d'erreur (headers already sent).
Cette fonction ouvre une session s'il n'existe pas de session déjà ouverte. Si une session a déjà été ouverte et que l'id de session a été correctement transmise à la page, elle reprend la session existante et permet l'accès aux variables de session correspond à cette session.
- **session_id([id])** : définit l'id de session à utiliser ou renvoie l'id de session utilisée. Lors d'un session_start, une id de session est automatiquement générée par PHP. Il n'est donc en général pas nécessaire de la définir.
- **session_name([nom])** : définit le nom de la session en cours ou renvoie le nom utilisé par la session en cours. Le nom par défaut est défini par le session.name défini dans la configuration de PHP.
- **session_register("variable")** : permet d'enregistrer la variable de session passée en paramètre. Il est à noter que pour enregistrer \$login, il faut utiliser session_register("login").
- **session_is_registered("variable")** : vérifie si la variable spécifiée en paramètre est enregistrée dans la session active.
- **session_unregister("variable")** : détruit une variable de session préalablement enregistrée.
- **session_destroy()** : détruit la session active.



Important : cette liste n'est pas exhaustive et ne dispense en aucun cas de la lecture du manuel présent sur php.net, rubrique Sessions.

Reprenons l'exemple vu précédemment.

Exemple 1 :

```
<?
session_start(); // Démarre la session. Spécifie un id de session
$login="nom"; // On définit la variable classique $login
$password="mot de passe"; // On définit la variable $password
session_register("login"); // On enregistre $login dans la session
session_register("password"); // On enregistre $password dans
// la session
header("Location: exemple2.php?".session_name()."."session_id());
exit(); // Redirection. On en reparle dans le II.3.
?>
```

Exemple 2 :

```

<?
// Ici $login et $password ne sont pas définis
session_start();
// Ils sont définis à partir d'ici.
echo $login;
echo $password;
if ($login != "nom") session_destroy(); // destruction de la session
?>

```

Cet exemple simple devrait vous permettre de comprendre comment sont manipulées les variables de session.

Si l'on souhaite modifier la valeur d'une variable de session, il faut procéder comme suit :

```

<?
session_start();
echo $login; // a la valeur de la variable de session correspondante
// la variable $login présente dans notre page prend une valeur
// différente de celle présente dans la session. La variable de
// session $login n'a pas été modifiée ; elle a toujours la même
// valeur !
$login="nouveau_login";
echo $login; // Affiche "nouveau_login"
// Met à jour la variable de session $login avec la nouvelle valeur à
// savoir "nouveau_login"
session_register("login");
?>

```

- Transmettre l'id de session

Comme nous l'avons vu précédemment, l'id de session permet d'identifier un utilisateur de manière unique et lui donne accès à ses variables de session. Il est donc nécessaire de transmettre cette id de session de page en page. Cette transmission doit être parfaite car si pour une page l'id de session n'est pas correctement transmise, le session_start crée une nouvelle session et les variables de session de la session précédente sont perdues.

Nous nous placerons ici dans un cadre général et ne rentrerons pas dans les cas particuliers de certains hébergeurs qui utilisent des versions spécifiques de PHP.

Voici les diverses solutions que l'on peut retenir. Il est à noter que l'utilisation de telle ou telle solution dépend à la fois de la configuration du serveur et de celle du client.

- La solution la plus simple : si PHP a été compilé avec l'option --enable-trans-sid et qu'on a configuré session.use_trans_sid à 1, il n'est pas nécessaire de se préoccuper de transmettre l'id de session, PHP la transmet automatiquement dans l'URL de manière transparente (on ne voit pas l'id de session dans l'URL). C'est sans doute la solution la plus élégante et la moins casse-tête. MAIS, cette solution ne peut pas être retenue dans la majorité des cas ! En effet, PHP ne gère plus cette fonctionnalité sitôt qu'on effectue une redirection avec un header ! De plus, si on utilise

des URL absolues, ce qui peut arriver dans certains développements, PHP ne transmet pas l'id de session car il suppose qu'on se dirige vers un site extérieur.

- On peut comme souvent se reposer sur un système de cookies. PHP le gère d'ailleurs automatiquement dans sa configuration avec `session.use_cookies`. S'il est à 1, l'id de session est automatiquement enregistré en tant que cookie sur l'ordinateur du client. Cette solution ne peut pas non plus être retenue telle quelle car si le client refuse les cookies, l'id de session n'est pas enregistrée.
- On peut décider de passer à chaque fois l'id de session dans l'URL. C'est une solution qui fonctionne à tous les coups même si elle est lourde et peu élégante.
Les URL seront alors de la forme : `"index.php?".session_name()."."session_id()` ou `index.php?<? echo session_name();?>=<? echo session_id(); ?>` ou autres variantes du même genre.

Ce sont les trois solutions que l'on peut retenir a priori. Il n'y a donc pas réellement de solutions miracles.

J'utilise personnellement une solution "batarde" qui fonctionne dans tous les cas et qui est, à mon sens, relativement légère.

Je me repose sur le `session.use_cookies=1`. Cependant, je prévois une solution de secours au cas où les cookies sont refusés par le client. J'utilise la constante `SID` définie par PHP. `SID` est une chaîne vide si l'id de session a bien été enregistrée dans un cookie et vaut `session_name()."."session_id()` si l'id de session n'a pas pu être enregistrée comme cookie.

Mes URL sont donc du type `"index.php?".SID` ou `index.php?<? echo SID; ?>` et autres variantes.

On peut d'ailleurs facilement écrire une fonction `url($chaîne)` qui ajoute `SID` ou non à la fin de l'URL passée en paramètre si `SID` est nul ou non. Ceci permet d'éviter les `?` et les `&` inutiles en fin d'URL.

NB : cette solution est une solution personnelle. Elle n'est pas parfaite et il existe d'autres méthodes qui fonctionnent tout aussi bien. A vous de trouver celle qui vous va le mieux.

• Sessions et programmation orientée objet

Les variables de session peuvent être de tout type : chaînes, entiers, tableaux, objets... Cependant, quand on utilise des objets, il est nécessaire de prendre quelques précautions : la définition de la classe doit être incluse avant le `session_start` afin que PHP puisse manipuler l'objet enregistré dans la session.

Exemple :

Supposons l'existence d'une classe `Utilisateur` définie dans le fichier `class_utilisateur.inc.php`. Supposons l'existence d'une variable de session `$user` qui est une instance d'`Utilisateur`. On écrira le début de notre fichier comme suit :


```

<?
// Inclusion de la définition de Utilisateur
include("class_utilisateur.inc.php");
// PHP connaît la définition de Utilisateur, il peut donc
// manipuler l'objet $user et récupérer les données stockées
// dans le fichier texte correspondant à la session.
session_start();
?>

```

A quoi ne servent pas les sessions ?

Les sessions ne sont pas à utiliser dans tous les cas. Il est parfois plus simple et plus élégant d'utiliser d'autres solutions. Les sessions ne s'appliquent pas de toute façon à toutes les situations.

Les sessions ne permettent pas de conserver des informations d'une visite à une autre. Pour cela, il faut soit recourir aux cookies pour enregistrer des données sur l'ordinateur du client (avec toujours les mêmes problèmes que précédemment), soit utiliser un système d'identification reposant sur une base de données.

Pour transmettre des informations vers une seule page, les sessions peuvent être lourdes à mettre en place. On peut alors soit recourir à un passage dans l'URL pour des informations courtes et peu sensibles, soit utiliser les champs cachés d'un formulaire du type `<input type="hidden" name="variable" value="valeur">`.

En conclusion, l'utilisation des sessions PHP4 peut être très intéressante mais elles ne sont pas à utiliser à tort et à travers. Leur utilisation est notamment très différente de l'utilisation des cookies.

Quelques erreurs communes

Ce paragraphe reprend quelques messages d'erreurs fréquents ainsi que les erreurs les plus souvent commises lors de l'utilisation des sessions. Cette liste est loin d'être exhaustive. N'hésitez pas à faire part de vos erreurs et du diagnostic correspondant dans les commentaires.

Erreur 1 :

Warning: Cannot send session cookie - headers already sent by (output started at c:\easyphp\www\test5.php:2) in c:\easyphp\www\test5.php on line 4

Warning: Cannot send session cache limiter - headers already sent (output started at c:\easyphp\www\test5.php:2) in c:\easyphp\www\test5.php on line 4

C'est l'erreur la plus classique. Un caractère affichable est présent avant le `session_start`. Ceci peut être un simple espace ou passage à la ligne avant le `<?` Ou un `echo` qui ne doit pas être avant le `session_start`.

Erreur 2 :

Fatal error: The script tried to execute a method or access a property of an incomplete object. Please ensure that the class definition utilisateur of the object you are trying to

operate on was loaded _before_ the session was started in
c:/easyphp/www/extranetpei/include/securite/fonction_check_auth_user.php on line 26

La classe d'une instance utilisée comme variable de session n'a pas été déclarée avant le session_start.
PHP n'arrive pas à gérer cette variable de session car il ne connaît pas la structure de l'objet.

Conclusion

Cet article n'a pas pour but d'être exhaustif ou de remplacer la documentation de PHP. Il s'agit juste de mettre en lumière ce que l'on peut faire et ne pas faire avec les sessions PHP4 ainsi que les problèmes que l'on peut rencontrer.

Il est loin d'être parfait ou complet et ne demande qu'à être amélioré.

ENITA Bordeaux
U.F. Informatique

Fonctions PHP

Gestion des chaînes de caractères : quelques fonctions

Fonctions	Description	Exemple
addslashes (\$chaîne)	Ajoute un slash devant tous les caractères spéciaux (' , " , \ , et NULL)	<code>\$x="L'école"; \$y = addslashes (\$x); // "L\'école"</code>
stripslashes (\$chaîne)	Enlève les slashes ajoutés par la fonction addslashes()	<code>\$y = "L\'école"; \$x = stripslashes (\$y); // "L'école"</code>
chop (\$chaîne) ou rtrim (\$chaîne)	Enlève les espaces de fin de chaîne (à droite =right).	<code>\$x = chop(" ENITA "); // " ENITA" \$x = rtrim(" ENITA "); // " ENITA"</code>
ltrim (\$chaîne)	Enlève les espaces de début de chaîne	<code>\$x = ltrim(" ENITA "); // "ENITA "</code>
trim (\$chaîne)	Enlève tous les espaces de début de chaîne et de fin de chaîne	<code>\$x = trim (" ENITA "); // "ENITA"</code>
chr (\$caractere)	Retourne le caractère correspondant au code ASCII donné	<code>\$x = chr (65); // "A"</code>
ord (\$caractere)	Retourne la valeur ASCII du caractère C'est l'inverse de la fonction chr()	<code>\$x = ord ("A"); // = 65</code>
explode (\$separ, \$chaîne, \$limit)	Scinde une chaîne en morceaux, grâce à un caractère délimiteur separ et met les valeurs dans un tableau. Si limit est fourni, le tableau retourné contiendra un maximum de limit éléments, et le dernier élément contiendra le reste de la chaîne.	<code>\$x = "ENITA Bordeaux"; \$mot = explode(" ", \$x, 2); // \$mot[0]="ENITA" // \$mot[1]="Bordeaux"</code>
implode (\$jointure, \$tableau) ou join (\$jointure, \$tableau)	Regroupe tous les éléments d'un tableau dans une chaîne, avec une chaîne de jointure.	<code>\$x[0]="ENITA"; \$x[1]="Bordeaux"; \$chaîne=implode("- ", \$x); // \$chaîne="ENITA - - Bordeaux"</code>
nl2br(\$chaîne)	Insère un retour ligne HTML avant chaque nouvelle ligne "\n"	<code>\$x = "ENITA\nBordeaux"; \$html = nl2br (\$x); // "ENITA
Bordeaux"</code>
strcasecmp(\$chaîne1, \$chaîne2)	Compare en binaire des chaînes, insensible à la casse. Retourne : < 0 si chaîne1 plus petite que chaîne2; > 0 si chaîne1 plus grande que chaîne2, = 0 si elles sont égales	<code>\$x1= "ENITA Bordeaux"; \$x2 = "enita bordeaux"; \$val=strcasecmp(\$x1,\$x2); // \$val = 0 (chaînes identiques)</code>
strcmp (\$chaîne1, \$chaîne2)	Identique à strcasecmp() mais cette fonction tient compte de la casse.	<code>\$x1= "ENITA Bordeaux"; \$x2 = "enita bordeaux"; \$val=strcmp(\$x1,\$x2); // < 0 valeur ascii de "E" < valeur ascii de "e"</code>
strip_tags (\$chaîne, \$tag)	Enlève les balises HTML et PHP	<code>\$x="<H1>ENITA</H1>"; \$y = strip_tags (\$x); // "ENITA"</code>

Fonctions	Description	Exemple
strlen (\$chaîne)	Retourne la longueur de la chaîne	<code>\$x = "ENITA Bordeaux"; \$lg = strlen (\$x); // = 14</code>
str_pad(\$chaîne, \$longueur, \$remplissage, \$type)	Complète une chaîne avec une autre. type peut prendre les valeurs de STR_PAD_RIGHT (par défaut), STR_PAD_LEFT, ou STR_PAD_BOTH.	<code>\$x = "ENITA Bordeaux"; \$y = str_pad (\$x, 20, "-="); // "ENITA Bordeaux---=-"</code>
strpos(\$chaîne, \$recherche, \$pos_debut)	Recherche la première occurrence d'un caractère dans une chaîne.	<code>\$x = "ENITA Bordeaux"; \$y = strpos (\$x, "de", 1); // = 10</code>
str_repeat (\$chaîne, \$nb)	Répète une chaîne	<code>\$x = str_repeat ("=-", 10); // "=-=-=-=-=-=-=-=-=-"</code>
strrev (\$chaîne)	Inverse l'ordre des caractères d'une chaîne	<code>\$x = "ENITA Bordeaux"; \$y = strrev (\$x); // xuaedroB ATINE</code>
strtolower(\$chaîne)	Met tous les caractères en minuscules	<code>\$x = "ENITA Bordeaux"; \$y = strtolower (\$x); // enita bordeaux</code>
strtoupper (\$chaîne)	Met tous les caractères en majuscules	<code>\$x = "ENITA Bordeaux"; \$y = strtoupper (\$x); // ENITA BORDEAUX</code>
str_replace(\$cherche, \$remplace, \$chaîne)	Remplace toutes les occurrences d'une chaîne par une autre.	<code>\$x = "ENITA BORDEAUX"; \$y = str_replace ("E", "W", \$x); // WNITA BORDWAUX</code>
substr (\$chaîne, \$debut, \$longueur)	Retourne une sous-chaîne	<code>\$x = "ENITA Bordeaux"; \$y = substr (\$x, 3, 5); // "ITA B"</code>
substr_count(\$chaîne, \$sous_chaîne)	Compte le nombre de sous-chaînes	<code>\$x = "ENITA Bordeaux"; \$y = substr_count(\$x, "or"); // = 1</code>
substr_replace(\$ss_chaîne, \$remplace, \$debut, \$longueur)	Remplace une sous-chaîne par une autre	<code>\$x = "ENITA BORDEAUX"; \$y = substr_replace (\$x, "DX", 8, 7); // "ORDEAUX" est remplacé par "DX" → "ENITA BDX"</code>

Fonctions mathématiques

Fonctions	Description	Exemple
abs(\$nb)	Retourne la valeur absolue d'un nombre	<code>\$x = abs(3.26) // x = 3.26 \$x = abs(-3.26) // x = 3.26</code>
ceil(\$nb)	Retourne l'entier supérieur ou égal à la valeur donnée en paramètre	<code>x = Math.ceil(6.01) // x = 7 x = Math.ceil(3.99) // x = 4</code>
Number_format(\$nb, \$nb_dec, \$sep_dec, \$sep_mil)	Met en forme \$nb avec le nombre de places décimales en utilisant les séparateurs décimal et des milliers.	<code>\$x = number_format(1006.01, 5, ",", " ") // x = 1 006,01000</code>
floor(\$nb)	Retourne l'entier inférieur ou égal à la valeur donnée en paramètre	<code>\$x = floor(6.01) // x = 6 \$x = floor(3.99) // x = 3</code>
round(\$nb)	Arrondit à l'entier le plus proche la valeur donnée en paramètre	<code>\$x = round(6.01) // x = 6 \$x = round(3.8) // x = 4 \$x = round(3.5) // x = 4</code>

Fonctions	Description	Exemple
max(\$nb1, \$nb2, ...)	Retourne le plus grand des nombres donnés en paramètre	<code>\$x = max(6, 7.25, -2.1) // x = 7.25</code> <code>\$x = max(-8.21, -3.65, -1) // x = -1</code> <code>\$x = max(5, 5) // x = 5</code>
min(\$nb1, \$nb2, ...)	Retourne le plus petit des nombres donnés en paramètre	<code>\$x = min(6, 7.25, 2.1) // x = 2.1</code> <code>\$x = min(-8.21, -3.65) // x = -8.21</code> <code>\$x = min(5, 5) // x = 5</code>
pow(\$val1, \$val2)	Retourne le nombre Val1 à la puissance Val2	<code>\$x = pow(3, 3) // x = 27</code> <code>\$x = pow(9, 0.5) // (racine carrée) x = 3</code>
rand(\$min, \$max)	Retourne un nombre aléatoire compris entre min et max	<code>\$x = rand(0, 1)</code> <code>// x = 0.6489534931546957</code>
srand(\$val)	Initialise le générateur de nombres aléatoires	<code>srand(12542) //</code>
Getrandmax()	Renvoie la plus grande valeur aléatoire pouvant être renvoyée par rand()	
mt_rand(\$min, \$max)*	Retourne un nombre aléatoire compris entre min et max (autre)	<code>\$x = mt_rand(0, 1)</code> <code>// x = 0.6489534931546957</code>
mt_srand(\$val)*	Initialise le générateur de nombres aléatoires	<code>mt_srand(12542) //</code>
mt_getrandmax()*	Renvoie la plus grande valeur pouvant être renvoyée par un appel à mt_rand()	
pi()	renvoie la valeur de Pi	
sqrt(\$nb)	Retourne la racine carrée du nombre passé en paramètre	<code>\$x = sqrt(9) // x = 3</code>
exp(\$nb)	Retourne l'exponentielle de la valeur nb	
log(\$nb)	Retourne le logarithme de la valeur nb	
log10(\$nb)	Retourne le logarithme de nb en base 10	
sin(\$nb)	Retourne le sinus de la valeur nb (donnée en radians)	
asin(\$nb)	Retourne l'arcsinus de la valeur nb (donnée en radians)	
cos(\$nb)	Retourne le cosinus de la valeur nb (donnée en radians)	
acos(\$nb)	Retourne l'arccosinus de la valeur nb (donnée en radians)	
tan(\$nb)	Retourne la tangente de la valeur nb (donnée en radians)	
atan(\$nb)	Retourne l'arctangente de la valeur nb (donnée en radians)	

* Ces fonctions utilisent les générateur de nombre aléatoire "Mersenne Twister" qui produit des nombres utilisables en cryptographie et qui est 4 fois plus rapide que la fonction standard (rand).